1. 4-Input AND / OR Gate

VHDL Code

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3  USE ieee.std_logic_arith.all;
4  USE ieee.std_logic_unsigned.all;
5
6
7  --entity
8
9  ENTITY val_4_and_or IS
10      PORT (
11
12 a,b,c,d : IN std_logic;
13           and_out, or_out : OUT std_logic
14      );
15 END ENTITY val_4_and_or;
16
17
18 --arcitecture
19 ARCHITECTURE bhv OF val_4_and_or IS
20
21 BEGIN
22      and_out <= a and b and c and d;
23      or_out <= a or b or c or d;
24
25
26 END ARCHITECTURE bhv;
```

Figure 1. VHDL code for the 4-input AND/OR gate

The circuit takes four 1-bit inputs a, b, c and d and outputs the AND and OR results at the same time using two concurrent signal assignments: and_out <= a and b and c and d, and or_out <= a or b or c or d.

Simulation Results

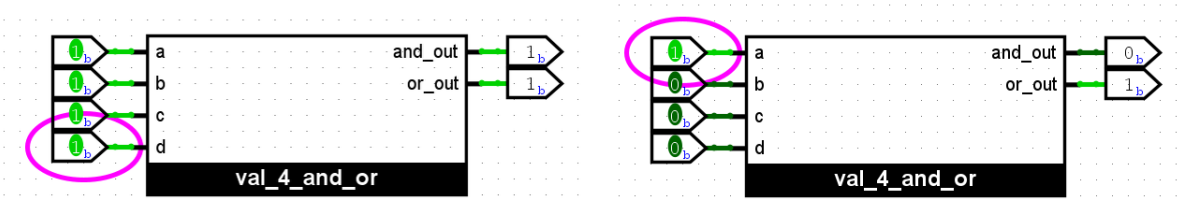


Figure 2. Outputs for inputs 1111 (left) and 1000 (right)

Input a b c d	and_out	or_out
1 1 1 1	1	1
1 0 0 0	0	1

Table 1. AND/OR gate simulation results

2. 8-to-1 Multiplexer (MUX)

VHDL Code

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4  --entity
5  ENTITY mux_8to1 IS
6      PORT (
7          a,b,c,d,e,f,g,h : IN std_logic;
8          ch0,ch1,ch2     : IN std_logic;
9          data : OUT std_logic
10     );
11 END ENTITY mux_8to1;
12
13 --architecture
14 ARCHITECTURE bhv OF mux_8to1 IS
15 BEGIN
16     PROCESS(a,b,c,d,e,f,g,h,ch0,ch1,ch2)
17     BEGIN
18         IF ch0 = '0' THEN
19             IF ch1 = '0' THEN
20                 IF ch2 = '0' THEN
21                     data <= a;    -- 000
22                 ELSE
23                     data <= b;    -- 001
24                 END IF;
25             ELSE
26                 IF ch2 = '0' THEN
27                     data <= c;    -- 010
28                 ELSE
29                     data <= d;    -- 011
30                 END IF;
31             END IF;
32         ELSE
33             IF ch1 = '0' THEN
34                 IF ch2 = '0' THEN
35                     data <= e;    -- 100
36                 ELSE
37                     data <= f;    -- 101
38                 END IF;
39             ELSE
40                 IF ch2 = '0' THEN
41                     data <= g;    -- 110
42                 ELSE
43                     data <= h;    -- 111
44                 END IF;
45             END IF;
46         END IF;
47     END PROCESS;
48 END ARCHITECTURE bhv;
```

Figure 3. VHDL code for the 8-to-1 MUX

One of the eight data inputs (a-h) is routed to the output data according to the select signals ch0, ch1 and ch2 (ch0 is the MSB). Nested IF statements inside a PROCESS select 000→a, 001→b, …, 111→h.

Simulation Results

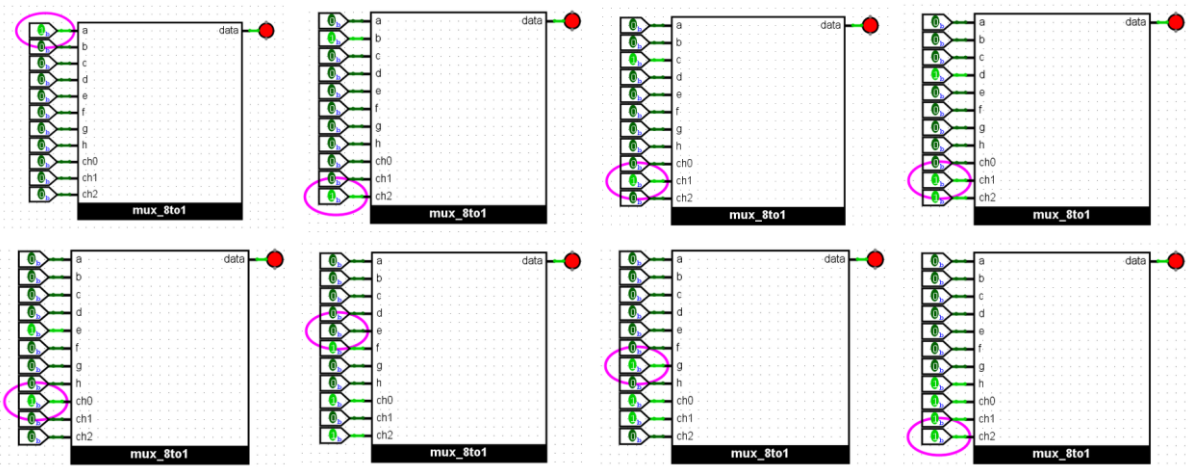


Figure 4. Output for select values 000-111 with only the selected input set to 1 (a-h)

ch0 ch1 ch2	Selected input	data
0 0 0	a	1
0 0 1	b	1
0 1 0	c	1
0 1 1	d	1
1 0 0	e	1
1 0 1	f	1
1 1 0	g	1
1 1 1	h	1

Table 2. 8-to-1 MUX simulation results

3. 7-Segment Character Decoder (A, b, C, d, F, g)

VHDL Code

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3  USE ieee.std_logic_arith.all;
4  USE ieee.std_logic_unsigned.all;
5
6
7  --entityx
8  ENTITY dis_7seg IS
9       PORT (
10           en   : IN std_logic;
11           data : IN std_logic_vector(3 DOWNTO 0);
12           dis  : OUT std_logic_vector(7 DOWNTO 0)
13      );
14 END ENTITY dis_7seg ;
15
16 --arcitecture
17 ARCHITECTURE bhv OF dis_7seg IS
18
19 BEGIN
20      PROCESS(en,data)
21      BEGIN
22           IF en = '0' THEN
23                dis <= "ZZZZZZZZ";
24           ELSE
25                CASE data IS
26                     WHEN "0000" => dis <= "11101110"; -- A
27                     WHEN "0001" => dis <= "00111110"; -- b
28                     WHEN "0010" => dis <= "10011100"; -- C
29                     WHEN "0011" => dis <= "01111010"; -- d
30                     WHEN "0100" => dis <= "10001110"; -- F
31                     WHEN "0101" => dis <= "11110110"; -- g
32
33                     WHEN others => dis <= "00000000";
34                END CASE;
35           END IF;
36      END PROCESS;
37
38 END ARCHITECTURE bhv;
```

Figure 5. VHDL code for the 7-segment decoder

A CASE statement decodes the 4-bit input data into the 8-bit segment signal dis.

Inputs 0000-0101 map to the patterns A, b, C, d, F and g, and all other values turn every segment off.

When en is 0, the output is high impedance ("ZZZZZZZZ").

Simulation Results

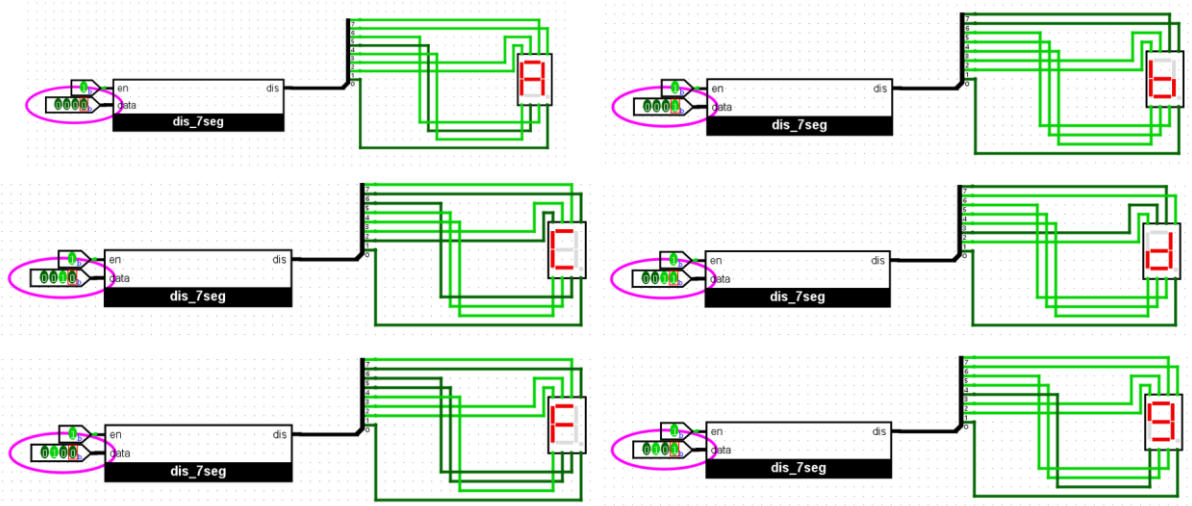


Figure 6. 7-segment display for data inputs 0000-0101

data	Displayed character
0000	A
0001	b
0010	C
0011	d
0100	F
0101	g

Table 3. 7-segment decoder simulation results

4. 25-Voter Voting Machine

VHDL Code

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3  USE ieee.std_logic_arith.all;
4  USE ieee.std_logic_unsigned.all;
5
6
7  --entityx
8  ENTITY vote25 IS
9       PORT (
10           vt : IN std_logic_vector(25 DOWNTO 1);
11           res : OUT std_logic
12      );
13 END ENTITY vote25;
14
15 --arcitecture
16 ARCHITECTURE bhv OF vote25 IS
17
18 BEGIN
19      PROCESS(vt)
20      VARIABLE sum : integer RANGE 0 TO 25;
21      BEGIN
22           sum := 0;
23           FOR i IN 1 TO 25 LOOP
24                IF (vt(i) = '1') THEN
25                     sum := sum+1;
26                     IF sum >= 13 THEN
27                          res <= '1';
28                     ELSE
29                          res <= '0';
30                     END IF;
31                END IF;
32           END LOOP;
33      END PROCESS;
34
35 END ARCHITECTURE bhv;
```

Figure 7. VHDL code for the 25-voter voting machine

Each bit of the 25-bit input vt(25 DOWNTO 1) is one vote, and a FOR loop counts the number of 1s (sum). If there are 13 or more yes votes (a majority), res = 1; otherwise res = 0.

Simulation Results

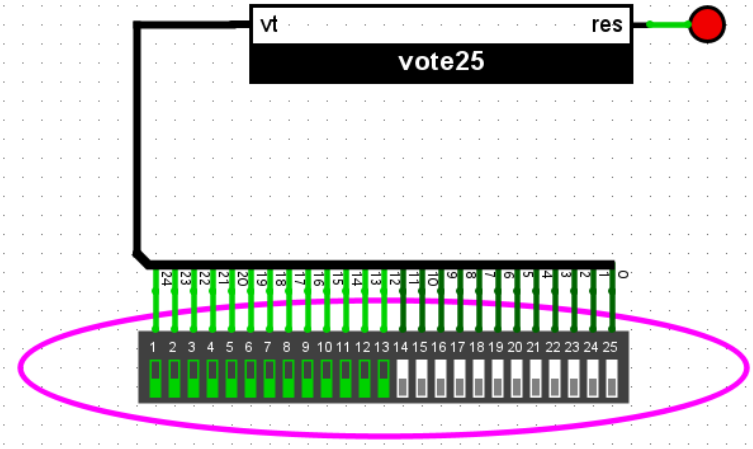


Figure 8. Output with 13 of 25 yes votes

With 13 DIP switches on, the output LED turned on, confirming that res = 1 when the majority (13 votes) is reached.

5. 15-Voter Voting Machine with 7-Segment Display (0-F)

VHDL Code

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_arith.all;
USE ieee.std_logic_unsigned.all;

--entity
ENTITY vote15_seg IS
     PORT (
          vt  : IN  std_logic_vector(14 DOWNTO 0);
          en  : IN  std_logic;
          seg : OUT std_logic_vector(6 DOWNTO 0)
     );
END ENTITY vote15_seg;

--architecture
ARCHITECTURE bhv OF vote15_seg IS
BEGIN
    PROCESS(vt, en)
        VARIABLE cnt : integer RANGE 0 TO 15;
    BEGIN
        cnt := 0;
        FOR i IN vt'RANGE LOOP
            IF vt(i) = '1' THEN
                cnt := cnt + 1;
            END IF;
        END LOOP;

        IF en = '0' THEN
            seg <= "0000000";
        ELSE
            CASE cnt IS
                WHEN 0  => seg <= "1111110"; -- 0
                WHEN 1  => seg <= "0110000"; -- 1
                WHEN 2  => seg <= "1101101"; -- 2
                WHEN 3  => seg <= "1111001"; -- 3
                WHEN 4  => seg <= "0110011"; -- 4
                WHEN 5  => seg <= "1011011"; -- 5
                WHEN 6  => seg <= "1011111"; -- 6
                WHEN 7  => seg <= "1110000"; -- 7
                WHEN 8  => seg <= "1111111"; -- 8
                WHEN 9  => seg <= "1111011"; -- 9
                WHEN 10 => seg <= "1110111"; -- A
                WHEN 11 => seg <= "0011111"; -- b
                WHEN 12 => seg <= "1001110"; -- C
                WHEN 13 => seg <= "0111101"; -- d
                WHEN 14 => seg <= "1001111"; -- E
                WHEN 15 => seg <= "1000111"; -- F
                WHEN OTHERS => seg <= "0000000";
            END CASE;
        END IF;
    END PROCESS;

END ARCHITECTURE bhv;
```

Figure 9. VHDL code for the 15-voter voting machine with 7-segment display

A FOR loop counts the 1s (cnt, 0-15) in the 15-bit input vt(14 DOWNTO 0), and a CASE statement outputs the 7-bit segment pattern seg.

Counts 0-9 are shown as digits and 10-15 as A, b, C, d, E and F. When en is 0, all segments are off.

Simulation Results

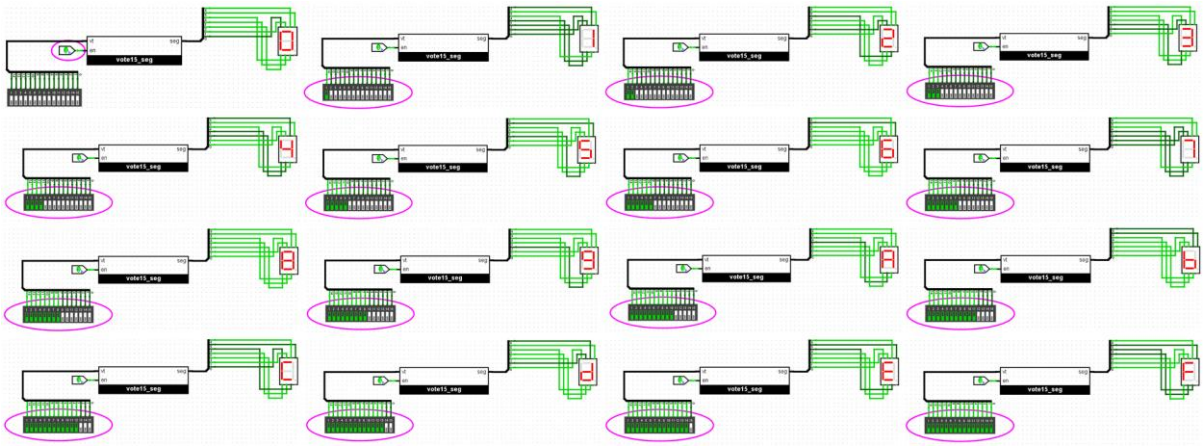


Figure 10. Display for 0-15 yes votes (0, 1, …, F from top left)